Unfortunately I wanted to focus on the portfolio so I'm just using the code from the github repository.

But with some additions by myself that are heavily based on existing code.

Performance

The Find() function operates on an unsorted array so it has to go through every element until if finds the correct one.

If it finds it on the first try it will be O(1) that time. If it has to go to the end of an array with n elements it will be O(n) that time.

The average will probably be around O(n/2). If we by n are referring to the amount of elements it has to go through it will always be O(n).

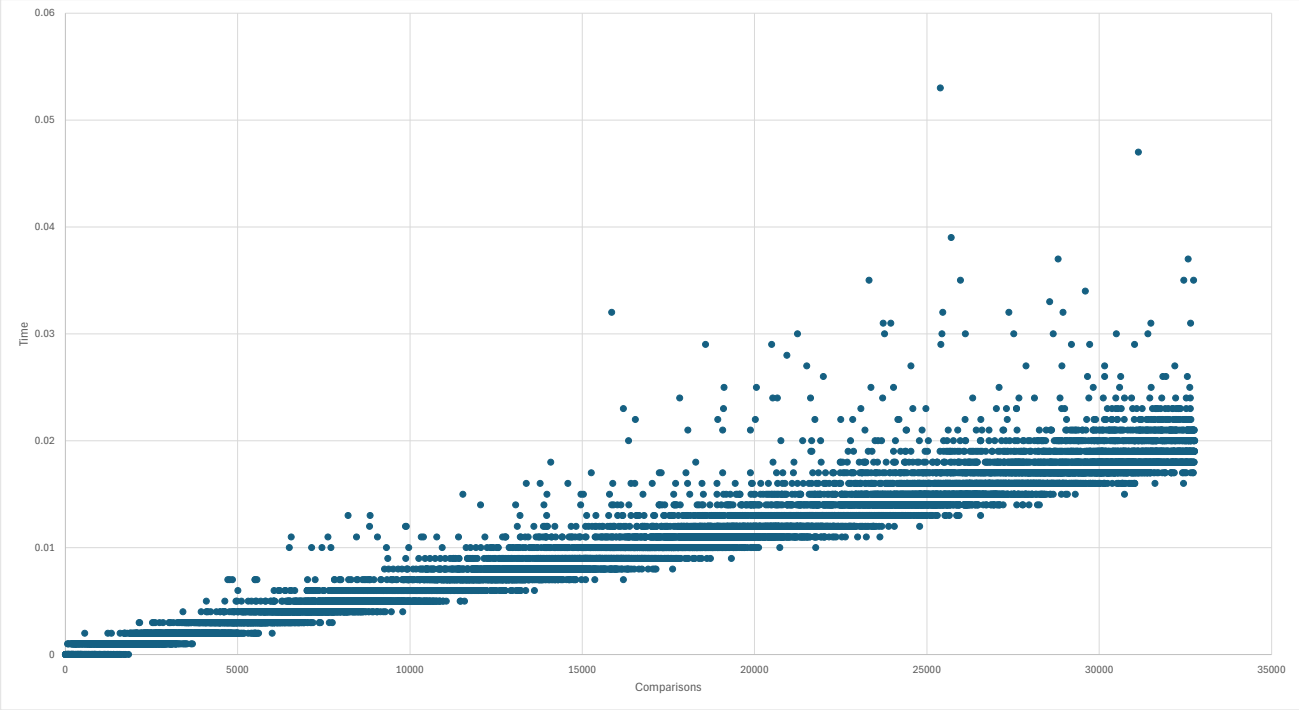
Here are some timings for the find functions when going through the whole list:

TLinkedList::Find():	0.026	0.026	0.027	0.032	0.026	0.028	0.032
FindAccountByNumber():	0.025	0.023	0.021	0.027	0.029	0.028	0.024

FindAccountByNumber() is faster, most likely because it moves a pointer ahead over an array that takes up contiguous memory

while TLinkedList::Find() has to move to random areas of memory all over the place

I decided to stress test Find() by doing 15 000 searches to get some data for a csv file that i then visualized:



We can see that the function clearly is O(n).

Flexibility

The PrintEveryAccountInDateRange() function iterates through the array and prints every element.

That means it has to implement iteration functionality in addition to the print functionality.

If you want to make another function that iterates over the array, you would most likely copy the function including the iteration logic and modify the rest.

This means that for every iterating function you would end up with copies of the same iteration logic.

If any changes are done to TLinkedList that breaks that logic you would have to change every iterating function.

What we have here is a classic example of high coupling.

TLinkedList::Every() on the other hand contains the logic for the iteration itself.

Authors of iterative functions only need to focus on what they have to do with every element, not how to get every element.

Changes in TLinkedList can be kept under damage control by ensuring that Every() remains a stable interface.

Other data structures can also provide an interface that has 1:1 external behavior with TLinkedList::Every().

That would allow the iterative functions to work interchangeably with the new data structure. Giving us high cohesion.